Lecture 2

Vector Quantized Variational Autoencoder

Introduction to Vector Quantized Variational Autoencoders (VQ-VAE)

🧩 What is VQ-VAE?

- VQ-VAE stands for **Vector Quantized Variational Autoencoder**.
- It introduces a **discrete latent space** into the autoencoder framework.
- Originally proposed by **van den Oord et al. (2017)** for image, audio, and symbolic modeling.

Introduction to Vector Quantized Variational Autoencoders (VQ-VAE)

🧩 What is VQ-VAE?

- VQ-VAE stands for **Vector Quantized Variational Autoencoder**.
- It introduces a **discrete latent space** into the autoencoder framework.
- Originally proposed by **van den Oord et al. (2017)** for image, audio, and symbolic modeling.

🔍 Why is it important?

- **VAEs** use continuous latent spaces → suited for smooth interpolation, but often generate **blurry** outputs.
- **VQ-VAEs** use a **finite set of learnable embeddings** (a codebook) to represent latent information → results in **sharper, more structured** reconstructions.
- Critical step toward **combining deep learning with discrete, symbolic reasoning** (used in DALL·E, Jukebox, etc.).

Introduction to Vector Quantized Variational Autoencoders (VQ-VAE)

2017-18

Neural Discrete Representation Learning

Aaron van den Oord
DeepMind
avdnoord@google.com

Oriol Vinyals
DeepMind
vinyals@google.com

Koray Kavukcuoglu
DeepMind
korayk@google.com

Motivation – Why Move from VAEs to VQ-VAEs?

Motivation – Why Move from VAEs to VQ-VAEs?

❌ Limitations of Standard VAEs (with continuous latent space)

- Latent vectors are sampled from a **Gaussian distribution**:

$$z \sim \mathcal{N}(0, I)$$

- Encourages **smooth interpolation**, but leads to:
  - 🌀 **Blurry reconstructions**
  - ❌ Difficulty modeling **sharp boundaries or discrete structure** (e.g. symbolic tokens, phonemes, characters)
  - 🚫 Ineffective in tasks requiring **categorical decisions or hierarchy**

Motivation – Why Move from VAEs to VQ-VAEs?

❌ Limitations of Standard VAEs (with continuous latent space)

- Latent vectors are sampled from a **Gaussian distribution**:

$$z \sim \mathcal{N}(0, I)$$

- Encourages **smooth interpolation**, but leads to:
  - 🌀 **Blurry reconstructions**
  - ❌ Difficulty modeling **sharp boundaries or discrete structure** (e.g. symbolic tokens, phonemes, characters)
  - 🚫 Ineffective in tasks requiring **categorical decisions or hierarchy**

✅ Why Discrete Latent Spaces?

- Many domains are **naturally discrete**:
  - **Language** → words, subwords, tokens
  - **Audio** → phonemes, units
  - **Images** → symbolic patches, parts
- Discrete spaces allow models to:
  - Learn **symbolic representations**
  - Combine with **autoregressive decoders** (e.g. Transformers)
  - Improve **generation sharpness** and **structure**

💡 The VQ-VAE Idea

> "Let's replace the continuous latent space with a **discrete codebook** of learnable embeddings."

- Each input is **quantized** to the **nearest vector** in a fixed-size dictionary.
- Model learns a set of **latent tokens**, like a neural vocabulary.

Architecture of VQ-VAE

🔄 Overview: Encoder → Quantizer → Decoder

1. Encoder

- Maps input x to continuous latent vector:

$$z_e(x) \in \mathbb{R}^D$$

- Think of this as a feature extractor.

Architecture of VQ-VAE

🔄 Overview: Encoder → Quantizer → Decoder

1. Encoder

- Maps input x to continuous latent vector:

$$z_e(x) \in \mathbb{R}^D$$

- Think of this as a feature extractor.

2. Vector Quantizer

- Uses a learnable **codebook** $\mathcal{E} = \{e_k\}_{k=1}^K$, where:
  - K: number of discrete codes (e.g. 512)
  - D: embedding dimension
- Replaces $z_e(x)$ with the **nearest codebook vector**:

$$z_q(x) = \text{Quantize}(z_e(x)) = e_k \text{ where } k = \arg\min_j \|z_e(x) - e_j\|_2$$

- Output is **discrete**, non-differentiable, but gradients are approximated.

Architecture of VQ-VAE

🔄 Overview: Encoder → Quantizer → Decoder

1. Encoder

- Maps input x to continuous latent vector:

$$z_e(x) \in \mathbb{R}^D$$

- Think of this as a feature extractor.

2. Vector Quantizer

- Uses a learnable **codebook** $\mathcal{E} = \{e_k\}_{k=1}^K$, where:
  - K: number of discrete codes (e.g. 512)
  - D: embedding dimension
- Replaces $z_e(x)$ with the **nearest codebook vector**:

$$z_q(x) = \text{Quantize}(z_e(x)) = e_k \text{ where } k = \arg\min_j \|z_e(x) - e_j\|_2$$

- Output is **discrete**, non-differentiable, but gradients are approximated.

3. Decoder

- Receives quantized embedding $z_q(x)$
- Reconstructs the original input:

$$\hat{x} = \text{Decoder}(z_q(x))$$

Vector Quantization Mechanism

🔤 The Codebook

- A set of **learnable embeddings** (a.k.a. prototypes or codewords):

$$\mathcal{E} = \{e_k \in \mathbb{R}^D\}_{k=1}^K$$

- Typically initialized randomly and learned during training.
- Think of it as a **vocabulary** of latent representations.

🔁 Quantization Step

- The encoder outputs a continuous vector $z_e(x) \in \mathbb{R}^D$.
- This is **quantized** to the nearest codebook entry:

$$z_q(x) = e_k \quad \text{where} \quad k = \arg\min_j \|z_e(x) - e_j\|_2$$

- This makes the latent representation **discrete**, yet expressive.

Vector Quantization Mechanism

🔤 The Codebook

- A set of **learnable embeddings** (a.k.a. prototypes or codewords):

$$\mathcal{E} = \{e_k \in \mathbb{R}^D\}_{k=1}^K$$

- Typically initialized randomly and learned during training.
- Think of it as a **vocabulary** of latent representations.

More on D and K later.....

🔁 Quantization Step

- The encoder outputs a continuous vector $z_e(x) \in \mathbb{R}^D$.
- This is **quantized** to the nearest codebook entry:

$$z_q(x) = e_k \quad \text{where} \quad k = \arg\min_j \|z_e(x) - e_j\|_2$$

- This makes the latent representation **discrete**, yet expressive.

Loss Function in VQ-VAE

🧠 **Goal: Train a discrete latent autoencoder to:**

- Reconstruct input well.
- Learn meaningful codebook embeddings.
- Ensure encoder commits to using the codebook entries.

📐 **Total Loss**

$$\mathcal{L} = \underbrace{\log p(x \mid z_q(x))}_{\text{Reconstruction}} + \underbrace{\|\text{sg}[z_e(x)] - e\|_2^2}_{\text{Codebook loss}} + \beta \underbrace{\|z_e(x) - \text{sg}[e]\|_2^2}_{\text{Commitment loss}}$$

Loss Function in VQ-VAE

🧠 **Goal: Train a discrete latent autoencoder to:**

- Reconstruct input well.
- Learn meaningful codebook embeddings.
- Ensure encoder commits to using the codebook entries.

📐 **Total Loss**

$$\mathcal{L} = \underbrace{\log p(x \mid z_q(x))}_{\text{Reconstruction}} + \underbrace{\|\text{sg}[z_e(x)] - e\|_2^2}_{\text{Codebook loss}} + \beta \underbrace{\|z_e(x) - \text{sg}[e]\|_2^2}_{\text{Commitment loss}}$$

🧭 `sg[·]` **stands for stop-gradient (or detach) operation.**

It's a mathematical notation that means:

> **"Pass the value forward during the forward pass, but block the gradient during backpropagation."**

more.....

🔁 Why is `sg[·]` used in VQ-VAE?

In the VQ-VAE loss:

$$\mathcal{L} = \log p(x|z_q(x)) + \|\text{sg}[z_e(x)] - e\|_2^2 + \beta\|z_e(x) - \text{sg}[e]\|_2^2$$

We use `sg[·]` in two places:

🔁 Why is `sg[·]` used in VQ-VAE?

In the VQ-VAE loss:

$$\mathcal{L} = \log p(x|z_q(x)) + \|\text{sg}[z_e(x)] - e\|_2^2 + \beta\|z_e(x) - \text{sg}[e]\|_2^2$$

We use `sg[·]` in two places:

📦 1. Codebook Loss

$$\|\text{sg}[z_e(x)] - e\|_2^2$$

- Here, we want to **update the codebook** e, to move closer to $z_e(x)$, but **don't want** to update the encoder from this term.
- So we apply `sg[z_e(x)]`.

🔁 Why is `sg[·]` used in VQ-VAE?

In the VQ-VAE loss:

$$\mathcal{L} = \log p(x|z_q(x)) + \|\text{sg}[z_e(x)] - e\|_2^2 + \beta\|z_e(x) - \text{sg}[e]\|_2^2$$

We use `sg[·]` in two places:

📦 1. Codebook Loss

$$\|\text{sg}[z_e(x)] - e\|_2^2$$

- Here, we want to **update the codebook** e, to move closer to $z_e(x)$, but **don't want** to update the encoder from this term.
- So we apply `sg[z_e(x)]`.

🧲 2. Commitment Loss

$$\|z_e(x) - \text{sg}[e]\|_2^2$$

- Here, we want to **update the encoder** to commit to a specific codebook entry, but **don't want** to update the codebook from this term.
- So we apply `sg[e]`.

🔁 Why is `sg[·]` used in VQ-VAE?

In the VQ-VAE loss:

$$\mathcal{L} = \log p(x|z_q(x)) + \|\text{sg}[z_e(x)] - e\|_2^2 + \beta\|z_e(x) - \text{sg}[e]\|_2^2$$

We use `sg[·]` in two places:

📦 1. Codebook Loss

$$\|\text{sg}[z_e(x)] - e\|_2^2$$

- Here, we want to **update the codebook** e, to move closer to $z_e(x)$, but **don't want** to update the encoder from this term.
- So we apply `sg[z_e(x)]`.

🧲 2. Commitment Loss

$$\|z_e(x) - \text{sg}[e]\|_2^2$$

- Here, we want to **update the encoder** to commit to a specific codebook entry, but **don't want** to update the codebook from this term.
- So we apply `sg[e]`.

🎯 Why this trick is useful:

Without `sg[]`, the encoder and codebook would interfere with each other's gradients — potentially collapsing or diverging during training.

With `sg[]`, we can **cleanly separate**:

- Encoder updates
- Decoder updates
- Codebook updates

📐 Total Loss

$$\mathcal{L} = \underbrace{\log p(x \mid z_q(x))}_{\text{Reconstruction}} + \underbrace{\|\text{sg}[z_e(x)] - e\|_2^2}_{\text{Codebook loss}} + \beta \underbrace{\|z_e(x) - \text{sg}[e]\|_2^2}_{\text{Commitment loss}}$$

📐 Total Loss

$$\mathcal{L} = \underbrace{\log p(x \mid z_q(x))}_{\text{Reconstruction}} + \underbrace{\|\text{sg}[z_e(x)] - e\|_2^2}_{\text{Codebook loss}} + \beta \underbrace{\|z_e(x) - \text{sg}[e]\|_2^2}_{\text{Commitment loss}}$$

📌 1. Reconstruction Loss

- Same as in standard autoencoders.
- Measures how well the decoder reconstructs the input from quantized latent $z_q(x)$.
- Usually implemented as **MSE** or **cross-entropy**.

📐 Total Loss

$$\mathcal{L} = \underbrace{\log p(x \mid z_q(x))}_{\text{Reconstruction}} + \underbrace{\|\text{sg}[z_e(x)] - e\|_2^2}_{\text{Codebook loss}} + \beta \underbrace{\|z_e(x) - \text{sg}[e]\|_2^2}_{\text{Commitment loss}}$$

📌 1. Reconstruction Loss

- Same as in standard autoencoders.
- Measures how well the decoder reconstructs the input from quantized latent $z_q(x)$.
- Usually implemented as **MSE** or **cross-entropy**.

📦 2. Codebook Loss

- Updates the codebook vectors to get closer to the encoder outputs:

$$\|\text{sg}[z_e(x)] - e\|_2^2$$

- Uses **stop-gradient (sg)** to prevent encoder from being updated by this term.

📐 Total Loss

$$\mathcal{L} = \underbrace{\log p(x \mid z_q(x))}_{\text{Reconstruction}} + \underbrace{\|\text{sg}[z_e(x)] - e\|_2^2}_{\text{Codebook loss}} + \beta \underbrace{\|z_e(x) - \text{sg}[e]\|_2^2}_{\text{Commitment loss}}$$

📌 1. Reconstruction Loss

- Same as in standard autoencoders.
- Measures how well the decoder reconstructs the input from quantized latent $z_q(x)$.
- Usually implemented as **MSE** or **cross-entropy**.

📦 2. Codebook Loss

- Updates the codebook vectors to get closer to the encoder outputs:

$$\|\text{sg}[z_e(x)] - e\|_2^2$$

- Uses **stop-gradient (sg)** to prevent encoder from being updated by this term.

🧲 3. Commitment Loss

- Encourages encoder outputs $z_e(x)$ to **commit** to a specific codebook entry.
- Prevents encoder from drifting too far from codebook:

$$\beta \|z_e(x) - \text{sg}[e]\|_2^2$$

- β is a hyperparameter, typically ~0.25.

Applications of VQ-VAE

🧠 Why is VQ-VAE impactful?

- Enables **discrete latent representations** — bridging the gap between neural networks and symbolic structure.
- Learns a **neural vocabulary**: latent space behaves like a set of tokens (e.g., subwords, parts, phonemes).
- Supports **autoregressive modeling** over discrete codes.

🖼️ 1. Image Compression

- VQ-VAE learns to **compress images** into a small grid of latent codes.
- Outperforms traditional autoencoders in **perceptual quality**.
- Example: Input 64×64 → latent grid 8×8 with 512 possible tokens.

> 🧠 Used as a learned equivalent of JPEG/PNG compression — but generative!

🎨 2. Image Generation (DALL·E, VQGAN)

- VQ-VAE provides discrete codebook → used as a vocabulary for **Transformer-based generation** (like GPT).
- **DALL·E 1**: Generates images from text using a VQ-VAE tokenizer.
- **VQGAN + CLIP**: Text-to-image guided synthesis with artistic freedom.

Applications of VQ-VAE

🔊 3. Audio Synthesis (VQ-VAE + WaveNet)

- In **Jukebox** (OpenAI), VQ-VAE encodes raw audio into discrete units.
- WaveNet or Transformer then models those code sequences.
- Allows realistic music generation in different styles and languages.

🗣️ 4. Speech and Text Modeling

- In TTS (Text-to-Speech), VQ-VAE helps model:
  - Phoneme-like structures
  - Style embeddings
- Discrete latent space is better aligned with **linguistic tokens**.

🎯 5. Robotics / Control

- Discretized latent space simplifies control policies.
- Easier to interpret symbolic states from sensory inputs.

VQ-VAE_6.ipynb

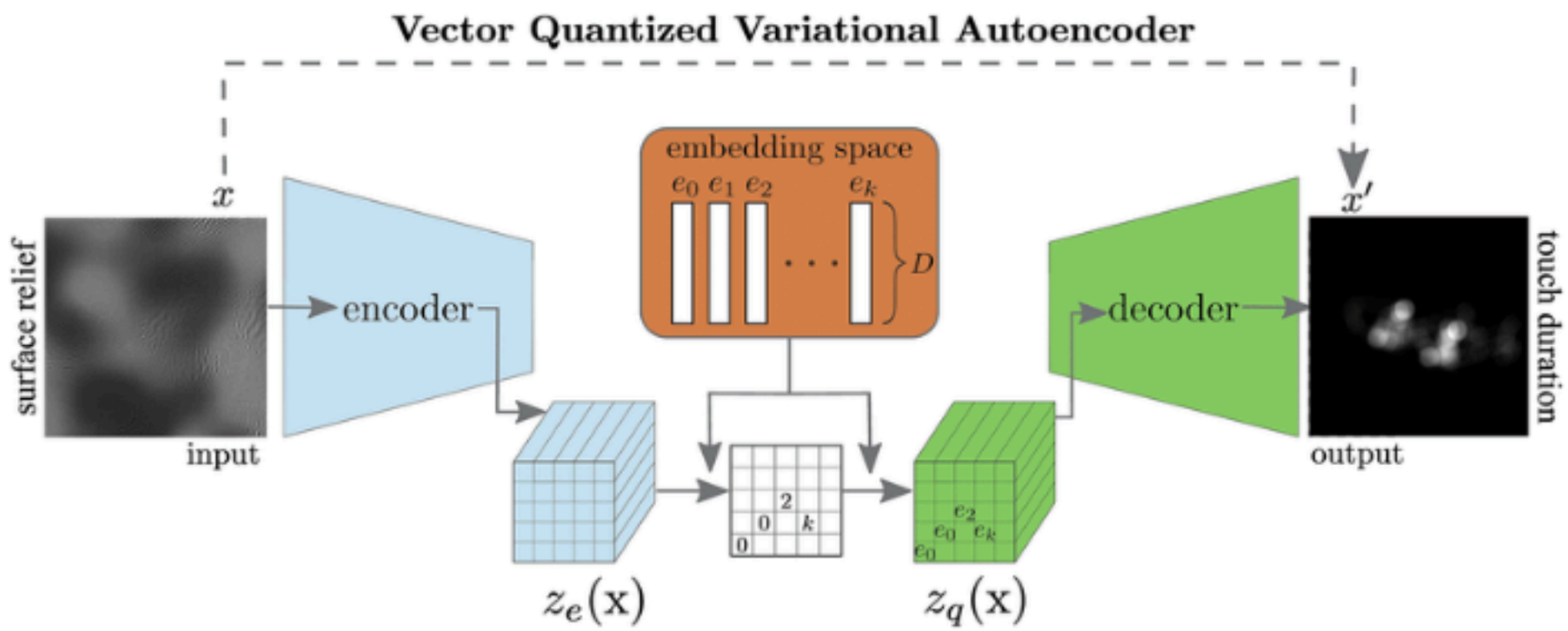
We move to Python implementation.....

🔍 Detailed Explanation of `VectorQuantizer` Class (VQ Layer)

python Copia Modifica

```python
class VectorQuantizer(nn.Module):
```

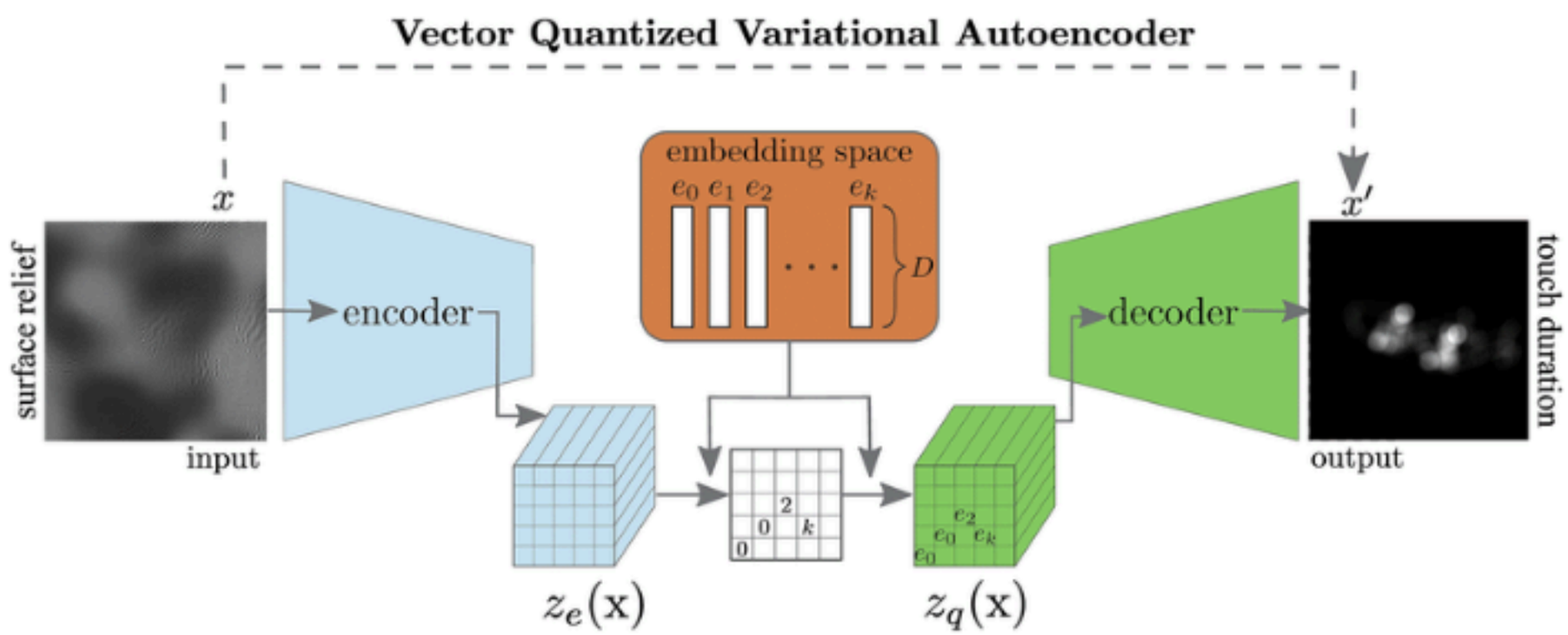
This defines the **core quantization module**. Its job is to replace each encoder output vector with the closest entry from a learned **codebook** (dictionary of size K).



🔧 `__init__`: Initialization

```python
def __init__(self, num_embeddings: int, embedding_dim: int, beta: float = 0.25):
```

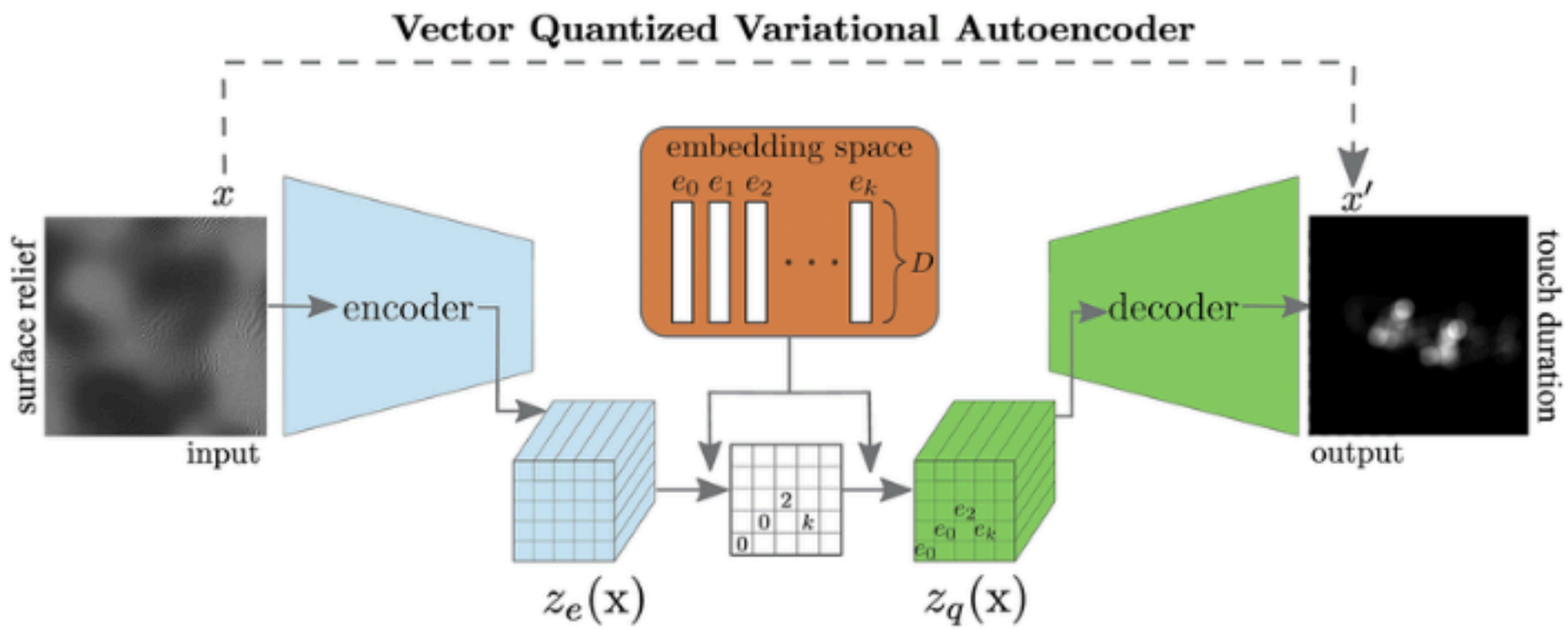
- `num_embeddings (K)`: how many discrete codebook entries (e.g., 512 or 1024)
- `embedding_dim (D)`: dimension of each vector in the codebook (e.g., 64 or 128)
- `beta`: weighting factor for the **commitment loss**, which ensures the encoder output stays close to the chosen codeword.



python Copia Modifica

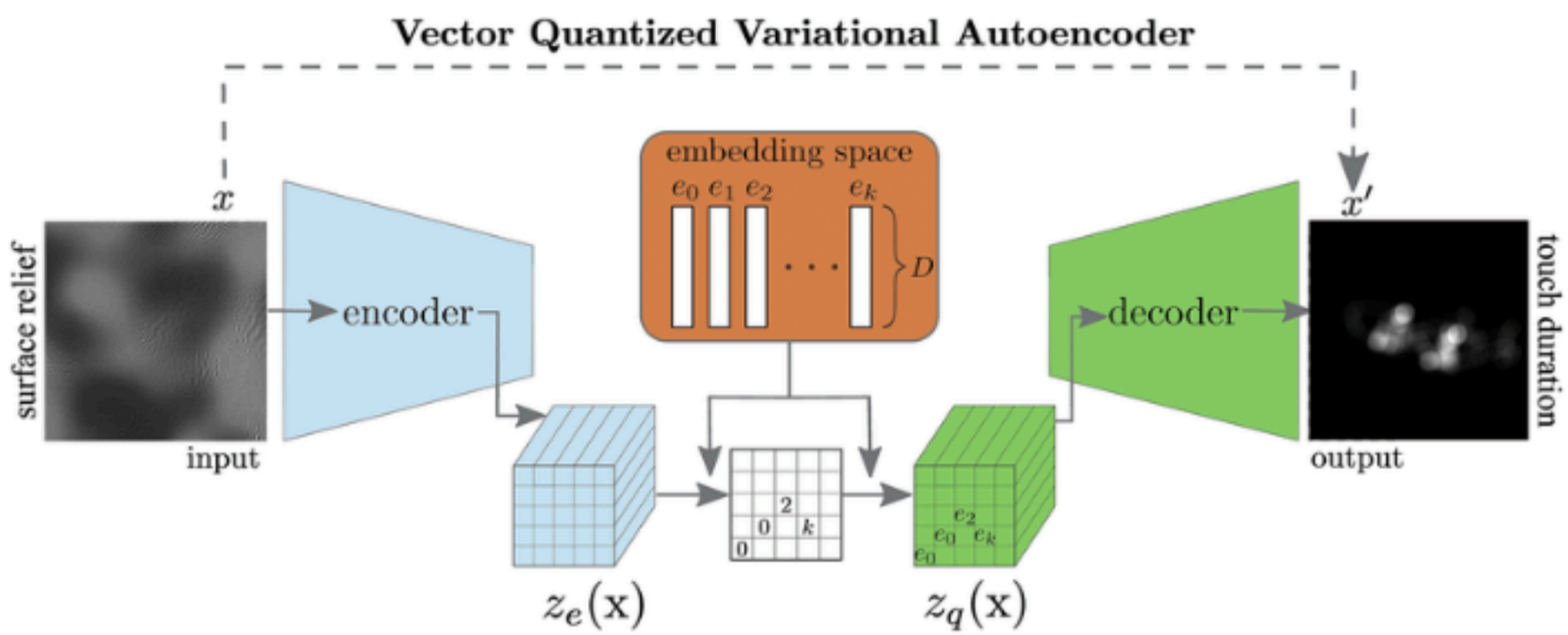
```python
super(VectorQuantizer, self).__init__()
self.K = num_embeddings
self.D = embedding_dim
self.beta = beta
```

Store the parameters.



```python
self.embedding = nn.Embedding(self.K, self.D)
self.embedding.weight.data.uniform_(-1 / self.K, 1 / self.K)
```

- `nn.Embedding` : learns a table of K embedding vectors, each of dimension D.
- Initialized with small uniform values.
  This is the **codebook** $\mathcal{E} = \{e_1, e_2, ..., e_K\} \in \mathbb{R}^{K \times D}$

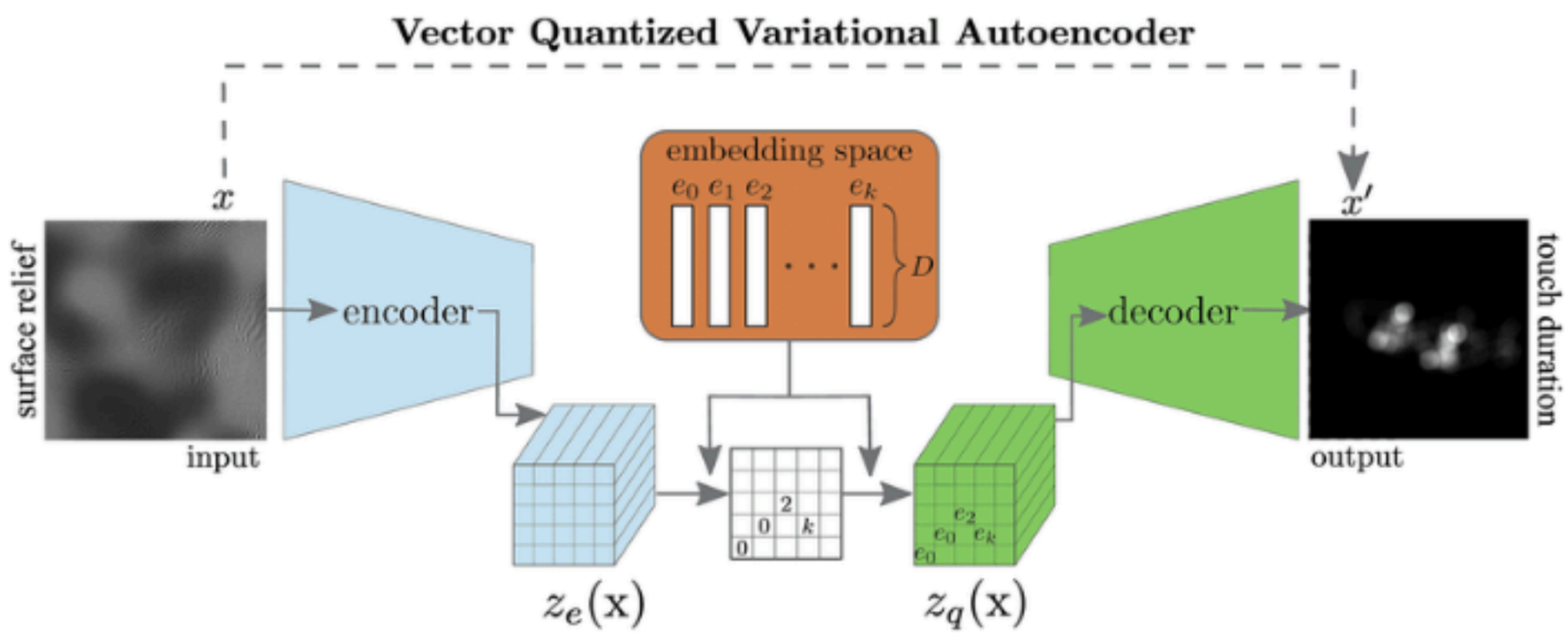


🚀 `forward`: Quantize Encoder Outputs

python Copia Modifica

```python
def forward(self, latents: torch.Tensor) -> tuple[torch.Tensor, torch.Tensor]:
```

`latents` is the output of the encoder, with shape `[B, D, H, W]`.

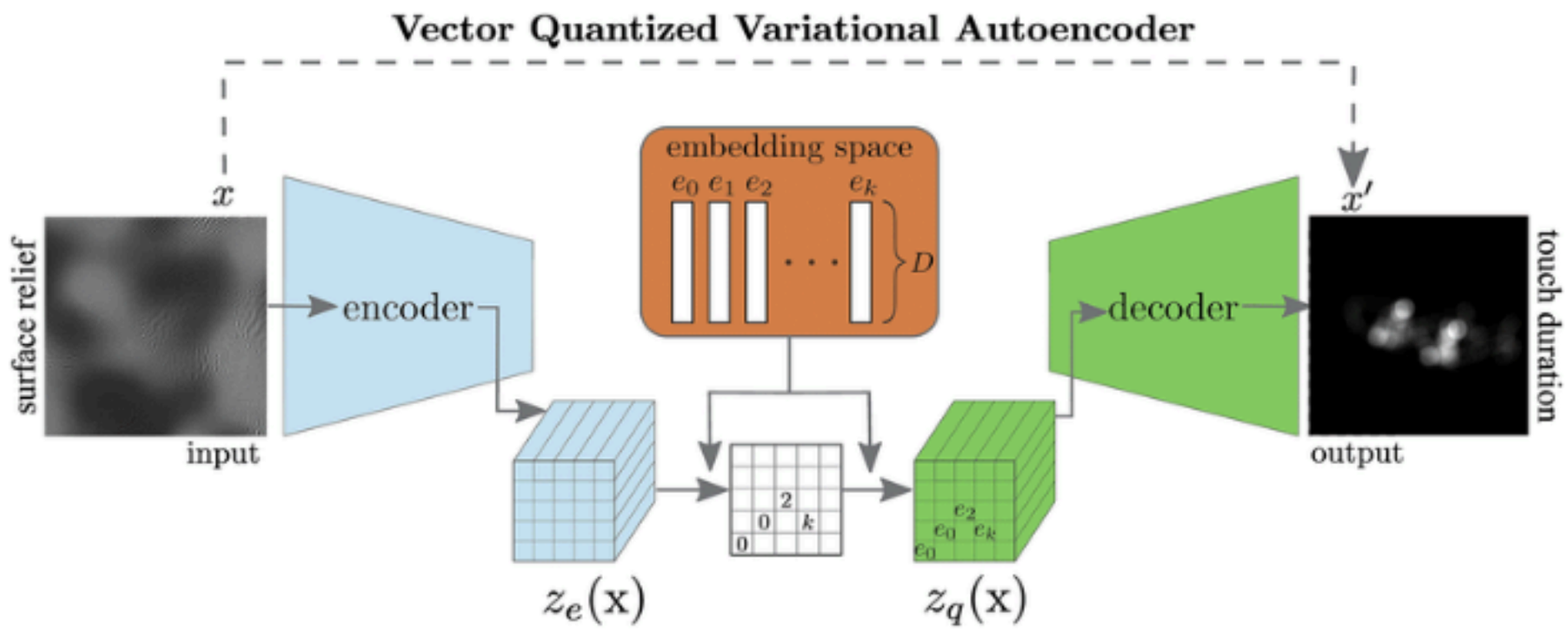


🔁 Step 1: Reshape to [BHW, D]

python Copia Modifica

```python
latents = latents.permute(0, 2, 3, 1).contiguous()  # [B, D, H, W] → [B, H, W, D]
latents_shape = latents.shape
flat_latents = latents.view(-1, self.D)  # [BHW, D]
```

Why? Because we'll quantize **each D-dimensional vector independently** at each spatial location. Flatten into one big batch of latent vectors.



✏️ Step 2: Compute Euclidean Distance to All Codebook Entries

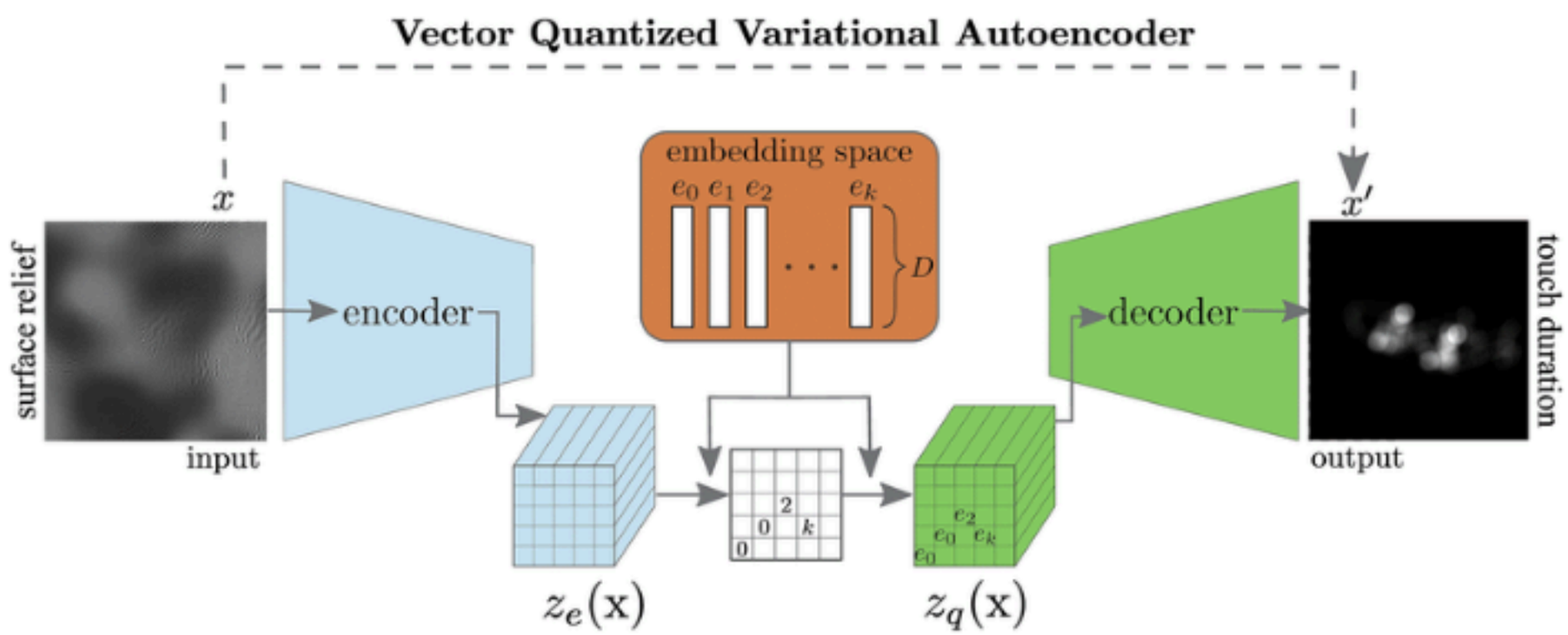
python Copia Modifica

```python
dist = (
    torch.sum(flat_latents ** 2, dim=1, keepdim=True) +  # ||z||²
    torch.sum(self.embedding.weight ** 2, dim=1) -       # ||e||²
    2 * torch.matmul(flat_latents, self.embedding.weight.t())  # -2zᵀe
)
```

This computes squared distance between **each latent vector and all K codewords** using the identity:

$$\|z - e_k\|^2 = \|z\|^2 + \|e_k\|^2 - 2z^T e_k$$

This gives a `[BHW, K]` tensor of distances.



🔢 Step 3: Find the Nearest Codeword Index

python Copia Modifica

```python
encoding_inds = torch.argmin(dist, dim=1).unsqueeze(1)  # [BHW, 1]
```

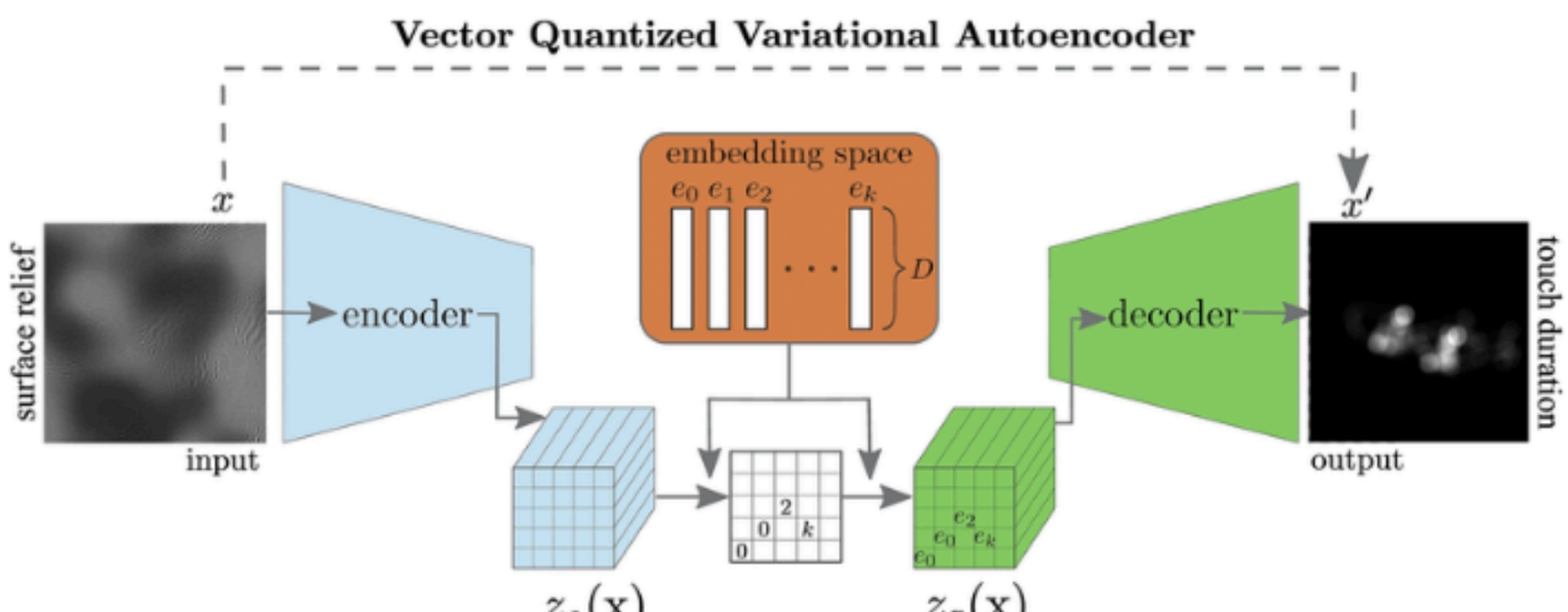
For each vector, pick the codebook index of the **closest embedding**.

🔠 Step 4: One-Hot Encoding of Indices

python Copia Modifica

```python
encoding_one_hot = torch.zeros(encoding_inds.size(0), self.K, device=device)
encoding_one_hot.scatter_(1, encoding_inds, 1)
```

Convert indices to one-hot vectors of shape `[BHW, K]`.

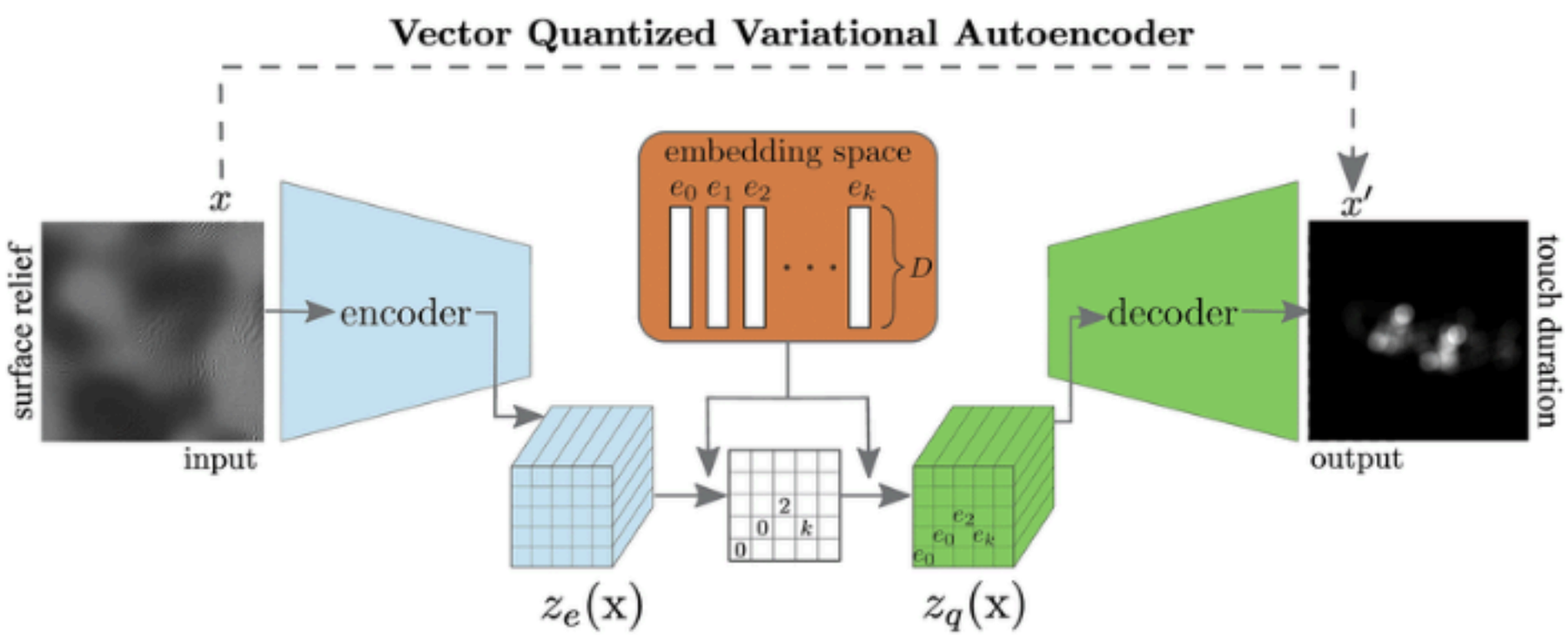


📌 Step 5: Lookup Quantized Embeddings

python Copia Modifica

```python
quantized = torch.matmul(encoding_one_hot, self.embedding.weight)  # [BHW, D]
quantized = quantized.view(latents_shape)  # [B, H, W, D]
```

This gives the **quantized version of the latents**, where each vector has been replaced by its nearest codeword.



🧱 Step 6: Compute VQ Loss

python Copia Modifica

```python
commitment_loss = F.mse_loss(quantized.detach(), latents)
embedding_loss = F.mse_loss(quantized, latents.detach())
vq_loss = commitment_loss * self.beta + embedding_loss
```

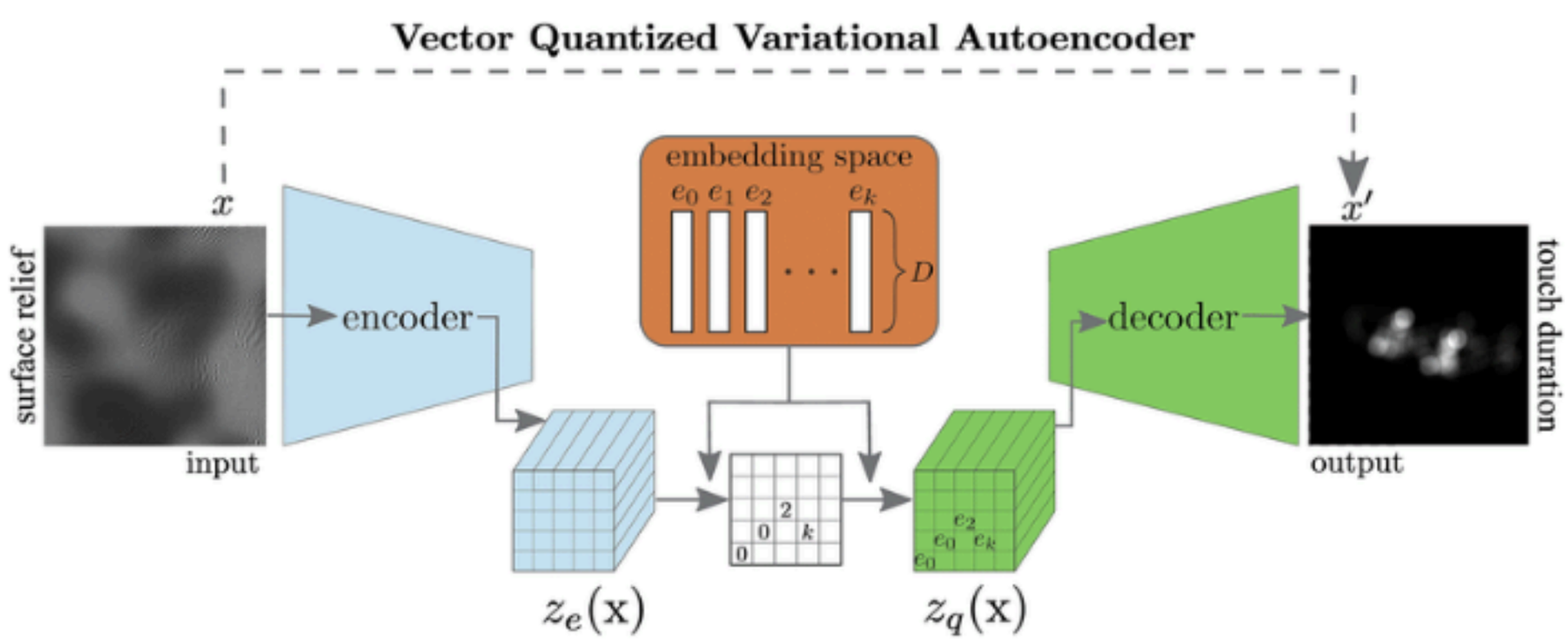
Two parts:

- `embedding_loss`: encourages the codebook vectors to move toward encoder output (gradient flows to codebook only)
- `commitment_loss`: encourages encoder output to stay close to selected codeword (gradient flows to encoder)

$$\boxed{\mathcal{L} = \underbrace{\log p(x \mid z_q(x))}_{\text{Reconstruction}} + \underbrace{\|\mathrm{sg}[z_e(x)] - e\|_2^2}_{\text{Codebook loss}} + \beta \underbrace{\|z_e(x) - \mathrm{sg}[e]\|_2^2}_{\text{Commitment loss}}}$$

Detaching is key:

- `.detach()` means **no gradients** will flow through that tensor.



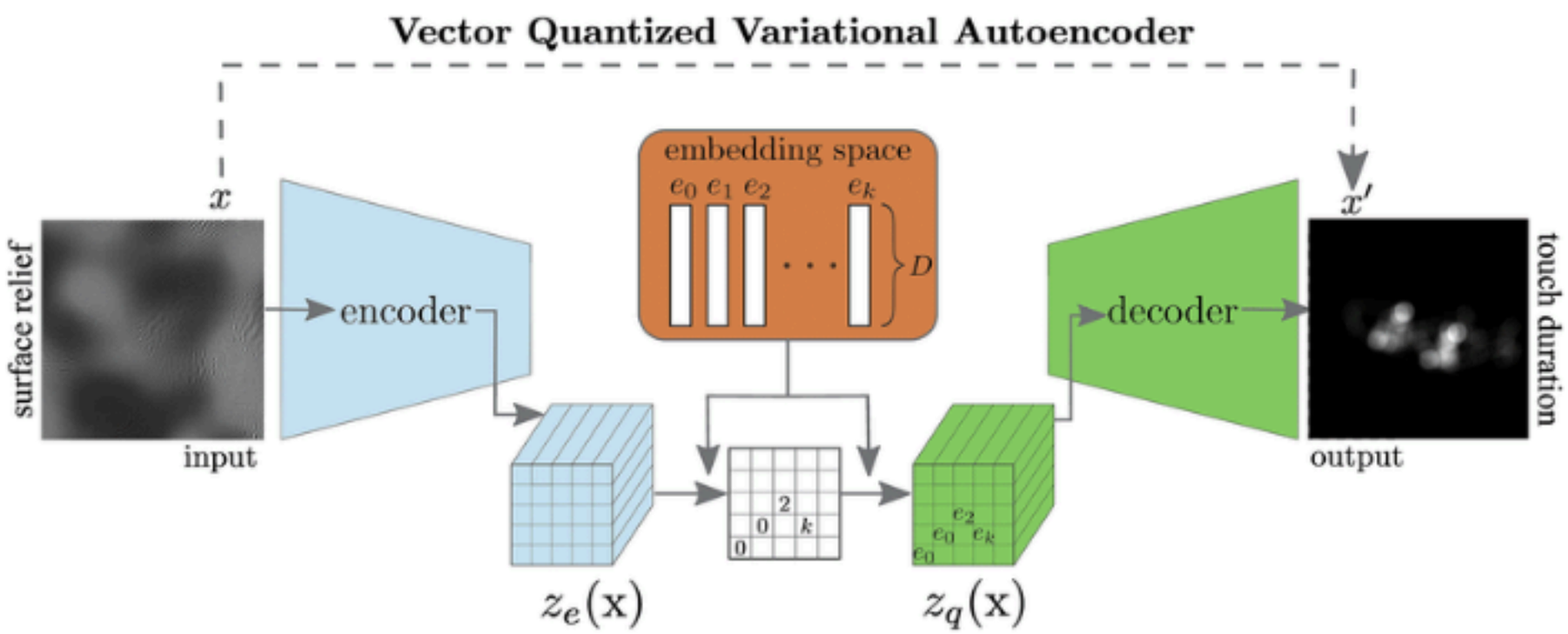
🔚 Step 8: Reshape Back to `[B, D, H, W]`

python Copia Modifica

```python
return quantized.permute(0, 3, 1, 2).contiguous(), vq_loss
```

We return:

- `quantized` : the quantized tensor (same shape as original input)
- `vq_loss` : the combined embedding + commitment loss



🔍 What Does the Decoder *Actually* See?

In **VQ-VAE**, the decoder is not given any continuous latent vector — instead, it sees a **grid of quantized codewords**:

- Shape: `[B, D, H, W]`, where `D` is the embedding dim
- Values: each vector is **chosen from the K codebook entries**

So at each of the $H \times W$ spatial locations, the model selects **1 out of K vectors**.

💡 Therefore:

The number of **possible latent configurations** (codebook lookups) is:

$$K^{H \times W}$$

⚠️ Important Clarification:

The decoder can generate **an enormous number of combinations** from these. Even with:

- $K = 512$
- $H = W = 16$

You have:

$$512^{256} \gg 10^{600}$$

So practically speaking, you're not limited in generation — it's a **discrete but *combinatorially huge*** latent space.

What Is a Residual Connection?

- Deep networks are hard to train: deeper → vanishing gradients.
- Residual learning addresses this by learning the **change** (residual) instead of the full function:

$$\text{Instead of:} \quad y = \mathcal{F}(x) \quad \text{We learn:} \quad y = x + \mathcal{F}(x)$$

- Introduced in **ResNet (2015)**, residual blocks allow **gradient flow through skip paths**.

🧠 **Intuition**: The block only needs to learn the difference from the identity — if no change is needed, it can just learn to return 0.

Residual Block Structure in VQ-VAE

The `ResidualLayer` used in the VQ-VAE encoder/decoder is minimal and efficient:

python Copia Modifica

```python
class ResidualLayer(nn.Module):
    def __init__(self, channels):
        self.block = nn.Sequential(
            nn.ReLU(True),
            nn.Conv2d(channels, channels, kernel_size=3, padding=1),
            nn.ReLU(True),
            nn.Conv2d(channels, channels, kernel_size=1)
        )

    def forward(self, x):
        return x + self.block(x)
```

🧩 What happens:

1. Input: $x \in \mathbb{R}^{C \times H \times W}$
2. Apply nonlinear transformation $f(x)$
3. Add original input: $y = x + f(x)$

✅ Output has **same shape** as input — this is key for identity mapping.

Block Internals Explained

Layer	Function	Why It's Used
`ReLU(True)`	Non-linearity with in-place computation	Efficient, keeps activations sparse
`Conv2d(3x3)`	Captures local features (preserves shape)	Core feature extractor
`ReLU(True)`	Adds non-linearity again	Introduces more expressive power
`Conv2d(1x1)`	Bottleneck projection (keeps size constant)	Compress or reshape features

🎯 The second conv (1×1) ensures minimal increase in computation while allowing deep feature learning.

Step-by-Step: Encoder with Residual Blocks

🔍 Full Encoder Block Explanation

python Copia Modifica

```python
# 🔧 Encoder: maps input images to a latent space representation

self.encoder = nn.Sequential(
    # 1️⃣ First conv: Downsample the image from 64x64 → 32x32
    nn.Conv2d(in_channels, hidden_channels // 2, kernel_size=4, stride=2, padding=1),
    nn.ReLU(),

    # 2️⃣ Second conv: Downsample again → 16x16
    nn.Conv2d(hidden_channels // 2, hidden_channels, kernel_size=4, stride=2, padding=
    nn.ReLU(),

    # 3️⃣ Third conv: keep same resolution (16x16), but deeper features
    nn.Conv2d(hidden_channels, hidden_channels, kernel_size=3, stride=1, padding=1),

    # 4️⃣ Residual blocks: refine the features, preserve resolution
    ResidualLayer(hidden_channels),
    ResidualLayer(hidden_channels)
)
```

Step-by-Step: Encoder with Residual Blocks

🔍 Full Encoder Block Explanation

python Copia Modifica

```python
# 🔧 Encoder: maps input images to a latent space representation

self.encoder = nn.Sequential(
    # 1️⃣ First conv: Downsample the image from 64x64 → 32x32
    nn.Conv2d(in_channels, hidden_channels // 2, kernel_size=4, stride=2, padding=1),
    nn.ReLU(),

    # 2️⃣ Second conv: Downsample again → 16x16
    nn.Conv2d(hidden_channels // 2, hidden_channels, kernel_size=4, stride=2, padding
    nn.ReLU(),

    # 3️⃣ Third conv: keep same resolution (16x16), but deeper features
    nn.Conv2d(hidden_channels, hidden_channels, kernel_size=3, stride=1, padding=1),

    # 4️⃣ Residual blocks: refine the features, preserve resolution
    ResidualLayer(hidden_channels),
    ResidualLayer(hidden_channels)
)
```

📐 Example: Shapes (assuming input is 64×64 RGB)

python Copia Modifica

```python
Input:            [B, 3, 64, 64]      # RGB image
After Conv1:      [B, 64, 32, 32]     # downsampled
After Conv2:      [B, 128, 16, 16]    # downsampled
After Conv3:      [B, 128, 16, 16]    # same
After Residuals:  [B, 128, 16, 16]    # same, but enriched
```

(assuming `hidden_channels = 128`)

🧠 Conceptual Breakdown

Layer	Operation	Output Size (H × W)	Purpose
`Conv2d(4×4, stride=2)`	Downsampling	64 → 32	Shrinks image while increasing channels
`Conv2d(4×4, stride=2)`	Downsampling	32 → 16	Further compress spatial info
`Conv2d(3×3, stride=1)`	Same size	16 → 16	Extracts finer features
`ResidualLayer × 2`	Identity + Nonlinear updates	16 → 16	Improves depth without destabilizing gradients

Motivation for ConvTranspose2d (in the Decoder...)

- In CNNs, `Conv2d` with `stride > 1` **reduces** spatial dimensions.
- In generative models (like VAEs), we need the **inverse** operation: to **increase** spatial dimensions.

Operation	Input Shape	Output Shape
`nn.Conv2d`	[B, C, 64, 64]	→ [B, C', 32, 32]
`nn.ConvTranspose2d`	[B, C, 32, 32]	→ [B, C', 64, 64]

Motivation for ConvTranspose2d (in the Decoder...)

- Think of it as a **reversed Conv2d**.
- Instead of applying a kernel sliding over the input, it **spreads** input values using the kernel.

🧱 Example: `ConvTranspose2d(kernel_size=3, stride=2, padding=1)`
→ roughly doubles the spatial size

📐 How it works:

- Inserts zeros between pixels
- Applies a learned kernel
- Produces **higher-resolution output**

🖼️ Intuitively:

mathematica Copia Modifica

```
Input (2x2)  →  Insert Zeros  →  Convolve  →  Output (4x4)
```

🧱 Step-by-Step VQ-VAE Model Architecture

We're combining three main components:

1. 🔽 **Encoder** – Compresses the input image into a latent representation.
2. 🎯 **Vector Quantizer** – Replaces latent vectors with nearest codebook entries.
3. 🔼 **Decoder** – Reconstructs the image from the quantized representation.

VQ-VAE_6.ipynb

Architecture Comparison

Component	VAE	VQ-VAE
Latent space	Continuous (Gaussian)	Discrete (Codebook of vectors)
Encoder Output	$\mu(x)$, $\log\sigma^2(x)$	Feature maps → quantized to nearest vector
Decoder Input	Sample $z \sim N(\mu, \sigma^2)$	Quantized embedding from codebook
Training Loss	ELBO = Recon + KL Divergence	Recon + Commitment + Codebook loss
Reparam. Trick	Required (for backprop)	Not needed – uses straight-through estimator

Loss Function Comparison

VAE Loss (ELBO):

$$\mathcal{L}_{\text{VAE}} = \mathbb{E}_{q(z|x)}[\log p(x|z)] - \text{KL}(q(z|x)\|p(z))$$

VQ-VAE Loss:

$$\mathcal{L}_{\text{VQ-VAE}} = \|x - \hat{x}\|^2 + \underbrace{\|\text{sg}[z_e(x)] - e\|^2}_{\text{Codebook Loss}} + \beta \underbrace{\|z_e(x) - \text{sg}[e]\|^2}_{\text{Commitment Loss}}$$

- `sg[·]` is stop-gradient operator (no gradients pass)
- Encourages:
  - Encoder to commit to codebook entries
  - Codebook to adapt to encoder's outputs

Applications and Strengths

Feature	VAE	VQ-VAE
Interpolation	Smooth & continuous	Discrete steps (possible jumps)
Compression	Limited by latent float precision	Excellent (z = indices)
Language + Vision	Not token-friendly	Tokenized latents → DALL·E, Transformers
GAN replacement	Less sharp outputs	Often used with PixelCNN or Transformers
Easy to train	KL balancing needed	Stable, but sensitive to codebook saturation

Use Cases and Models

- **VAE**:
  - Basic generative models
  - Semi-supervised learning
  - Latent space explorations (e.g., style transfer)
- **VQ-VAE**:
  - DALL·E 1: Image generation from text
  - VQ-GAN: High-res image synthesis
  - Video compression (e.g., Google DeepMind)
  - Discrete world models